

pandas를 활용한 데이터 분석

[1차시] Series와 DataFrame 만들기

📖 핵심 개념 설명

pandas는 행과 열로 구성된 표 데이터를 다루는 파이썬 라이브러리입니다. Series는 이름표가 있는 1차원 자료이고 DataFrame은 여러 열로 이루어진 표입니다. 리스트와 딕셔너리로 작은 설문 표를 직접 만들어 두 구조를 비교합니다. 실행 환경에 pandas가 없다면 터미널에서 `python -m pip install pandas matplotlib`를 한번 실행합니다.

📄 전체 실습 코드

```
import pandas as pd

scores = pd.Series([82, 75, 90], name="score")
data = {
    "brand": ["세종우유", "한솔제과", "세종푸드"],
    "score": [82, 75, 90],
    "responses": [20, 18, 25]
}
df = pd.DataFrame(data)

print("한 열의 자료:")
print(scores)
print("전체 표:")
print(df)
print("행과 열의 개수:", df.shape)
```

🔍 코드 토크아보기

- **import pandas as pd** : pandas를 pd라는 짧은 이름으로 사용합니다. 각 차시는 이 구문부터 독립적으로 실행할 수 있습니다.
- **pd.DataFrame(data)** : 딕셔너리의 키는 열 이름이 되고 각 리스트는 열의 값이 됩니다. 열마다 값의 개수가 같아야 합니다.
- **df.shape** : (3, 3)은 3행 3열이라는 뜻입니다. 왼쪽의 기본 인덱스 0, 1, 2는 별도 데이터 열이 아닙니다.

[2차시] CSV 파일 저장과 불러오기

📖 핵심 개념 설명

CSV는 쉼표로 열을 구분하는 텍스트 파일 형식입니다. pandas의 read_csv로 파일을 표로 읽고 to_csv로 분석 결과를 저장할 수 있습니다. 외부 파일 없이 실습하도록 예제 CSV를 먼저 만든 뒤 다시 읽습니다. 아래 코드는 현재 작업 폴더에 lesson02_survey.csv를 만들며 같은 이름이 있으면 덮어씁니다.

📄 전체 실습 코드

```
import pandas as pd

sample = pd.DataFrame({
    "brand": ["세종우유", "한솔제과", "세종푸드"],
    "score": [82, 75, 90]
})
file_name = "lesson02_survey.csv"
sample.to_csv(file_name, index=False, encoding="utf-8")

df = pd.read_csv(file_name, encoding="utf-8")
print("읽어 온 자료:")
print(df.head())
print("열 이름:", df.columns)
print("행 수:", len(df))
```

🔍 코드 튜아보기

- **to_csv와 index=False** : 표를 파일로 저장하면서 행 인덱스를 별도 열로 쓰지 않게 합니다.
- **encoding="utf-8"** : 한글 CSV의 인코딩을 지정합니다. 실제 파일은 저장된 인코딩에 맞추어 읽어야 합니다.
- **read_csv와 경로** : 같은 작업 폴더의 파일을 읽습니다. FileNotFoundError가 나면 파일 이름과 경로를 확인합니다. head()는 기본적으로 처음 5행까지 보여줍니다.

[3차시] 자료 구조와 기초 통계 확인

📖 핵심 개념 설명

분석을 시작하기 전에는 행과 열의 크기, 자료형, 누락값 유무를 먼저 살펴야 합니다. `info`는 열별 자료형과 값이 있는 개수를 보여주고 `describe`는 숫자 열의 기초 통계를 요약합니다. 수치만 보고 결론을 내리기 전에 자료가 어떤 상태인지 확인하는 습관을 기릅니다.

📄 전체 실습 코드

```
import pandas as pd

df = pd.DataFrame({
    "brand": ["A", "B", "C", "D"],
    "score": [80, 90, None, 70],
    "responses": [20, 30, 10, 20]
})

print("자료 일부:")
print(df.head(3))
print("크기:", df.shape)
df.info()
print("숫자 열 통계:")
print(df.describe())
print("열별 누락값 수:")
print(df.isna().sum())
```

🔍 코드 톡아보기

- `df.info()` : 자료형과 Non-Null Count를 확인합니다. `info` 자체가 정보를 출력하므로 `print(df.info())`로 감싸지 않습니다.
- `describe()` : count, mean, std, min, 사분위수, max를 보여줍니다. `score`의 유효 개수는 3이고 평균은 80입니다.
- `isna().sum()` : 누락 여부를 True/False로 표시한 뒤 열별로 합산합니다. `score`에만 누락값이 1개 있습니다.

[4차시] 열 선택과 loc·iloc 이해하기

📖 핵심 개념 설명

DataFrame에서 필요한 부분을 고를 때는 이름표로 선택하는 loc와 정수 위치로 선택하는 iloc를 구분해야 합니다. 열 하나는 Series로, 여러 열은 DataFrame으로 꺼낼 수 있습니다. 행 인덱스를 문자로 지정하여 이름과 위치의 차이를 분명하게 확인합니다.

📄 전체 실습 코드

```
import pandas as pd

df = pd.DataFrame({
    "brand": ["세종우유", "한솔제과", "세종푸드"],
    "score": [82, 75, 90],
    "responses": [20, 18, 25]
}, index=["a", "b", "c"])

print(df["score"])
print(df[["brand", "score"]])
print("이름표로 선택:")
print(df.loc["a":"b", ["brand", "score"]])
print("위치로 선택:")
print(df.iloc[0:2, 0:2])
```

🔍 코드 토크아보기

- **한 겹과 두 겹 대괄호** : df["score"]는 Series입니다. 여러 열 이름을 리스트로 전달한 df[["brand", "score"]]는 DataFrame입니다.
- **loc["a":"b"]** : 이름표 범위의 끝 b를 포함합니다. 첫 두 행의 지정한 열을 선택합니다.
- **iloc[0:2, 0:2]** : 정수 위치의 끝 2는 제외합니다. 앞의 범위는 행, 뒤의 범위는 열이므로 첫 두 행과 첫 두 열을 선택합니다.

[5차시] 조건 필터링과 정렬

📖 핵심 개념 설명

분석 질문을 조건식으로 바꾸면 관심 있는 행만 추출할 수 있습니다. pandas의 여러 조건은 and 대신 & 또는 |로 결합하고 각 조건을 괄호로 감쌉니다. 점수가 높고 응답 수가 충분한 브랜드를 고른 뒤 점수순으로 정리합니다.

📄 전체 실습 코드

```
import pandas as pd

df = pd.DataFrame({
    "brand": ["A", "B", "C", "D"],
    "score": [85, 72, 90, 88],
    "responses": [25, 30, 12, 22]
})

condition = (df["score"] >= 80) & (df["responses"] >= 20)
selected = df.loc[condition, ["brand", "score", "responses"]]
ranked = selected.sort_values("score", ascending=False)

print("조건에 맞는 행:")
print(ranked)
print("선택한 행 수:", len(ranked))
```

🔍 코드 뜯아보기

- **조건식의 결과** : 각 행마다 참과 거짓이 담긴 Series를 만듭니다. &는 두 조건이 모두 참인 행을 고릅니다.
- **loc로 필터링** : 85점·25명인 A와 88점·22명인 D가 선택됩니다. C는 점수가 높아도 응답 수 조건을 통과하지 못합니다.
- **sort_values** : ascending=False는 내림차순입니다. D, A 순서로 출력되며 원래 df를 직접 바꾸지 않습니다.

[6차시] 자료형 변환과 결측·중복 처리

📌 핵심 개념 설명

실제 자료에는 숫자처럼 보이는 문자열, 잘못 입력한 값, 누락값, 중복 행이 섞여 있을 수 있습니다. 숫자로 변환할 수 없는 값을 결측으로 바꾸고 중복을 제거한 뒤 분석 가능한 행을 선택합니다. 결측값을 무조건 0으로 채우지 않고 처리 기준과 제외 개수를 함께 기록합니다.

📄 전체 실습 코드

```
import pandas as pd

df = pd.DataFrame({
    "id": [1, 2, 2, 3, 4],
    "score": ["80", "90", "90", "미응답", None]
})

clean = df.drop_duplicates().copy()
clean["score"] = pd.to_numeric(clean["score"], errors="coerce")
print("결측값 수:", clean["score"].isna().sum())
valid = clean.dropna(subset=["score"]).copy()

# 평균 대체는 비교용
filled = clean["score"].fillna(clean["score"].mean())
print("평균으로 대체한 경우:", filled.tolist())
print("원본 행 수:", len(df), "유효 행 수:", len(valid))
print("유효 점수 평균:", valid["score"].mean())
```

🔍 코드 톺아보기

- **drop_duplicates**: 모든 열이 같은 중복 행 하나를 제거합니다. id만 같다고 무조건 중복으로 삭제하는 것은 아닙니다.
- **to_numeric(..., errors="coerce")**: 미응답 같은 문자열을 결측값으로 바꿉니다. 기존 None과 함께 결측값은 2개가 됩니다. 숫자로 바꿀 수 없는 비정상적인 값(미응답)을 발견했을 때 에러를 내는 대신 강제로 NaN(결측값)으로 변환한다.
- **dropna**와 **fillna**: 분석에는 결측 행을 제외한 80과 90을 써 평균 85를 구합니다. 평균 대체는 분포를 바꾸므로 목적에 맞게 선택해야 합니다.

[7차시] 파생 열 만들기과 요약 통계

📌 핵심 개념 설명

기존 열을 계산하여 새 열을 만들면 원래 자료에 없던 비교 기준을 얻을 수 있습니다. 광고를 본 사람 중 구매 의사를 밝힌 사람의 비율을 계산하고 요약합니다. 비율은 분모가 0이면 정의할 수 없으므로 해당 행을 먼저 제외합니다.

📄 전체 실습 코드

```
import pandas as pd

df = pd.DataFrame({
    "brand": ["A", "B", "C", "D"],
    "viewers": [100, 80, 120, 0],
    "buyers": [25, 24, 30, 0]
})

valid = df.loc[df["viewers"] > 0].copy()
valid["intent_rate"] = valid["buyers"] / valid["viewers"] * 100
valid["intent_rate"] = valid["intent_rate"].round(1)

print(valid)
print("브랜드별 비율의 단순 평균:", valid["intent_rate"].mean())
weighted_rate = valid["buyers"].sum() / valid["viewers"].sum() * 100
print("전체 응답자 기준 비율:", round(weighted_rate, 1))
```

🔍 코드 톡아보기

- **열 단위 계산** : 각 행의 buyers를 같은 행의 viewers로 나눕니다. 별도 for 반복문 없이 전체 열을 계산합니다.
- **copy와 새 열** : 필터링 결과를 복사한 뒤 intent_rate 열을 추가합니다. 비율은 A 25%, B 30%, C 25% 입니다.
- **평균의 해석** : 브랜드 비율의 단순 평균은 약 26.7%, 전체 응답자 기준 비율은 약 26.3% 입니다. 표본 수가 다르면 두 값은 다를 수 있습니다.

[8차시] groupby로 그룹별 분석하기

핵심 개념 설명

전체 평균만 보면 브랜드나 매체 사이의 차이를 놓칠 수 있습니다. groupby는 같은 값을 가진 행을 묶고 각 그룹에 집계 함수를 적용합니다. 브랜드별 응답 수와 평균 점수를 계산하고 매체별 결과와 비교하며 분석 질문에 맞는 묶음 기준을 선택합니다.

전체 실습 코드

```
import pandas as pd

df = pd.DataFrame({
    "brand": ["A", "A", "B", "B", "C", "C"],
    "media": ["영상", "포스터", "영상", "포스터", "영상", "포스터"],
    "score": [80, 90, 70, 80, 85, 95]
})

summary = df.groupby("brand", as_index=False).agg(
    responses=("score", "count"),
    average=("score", "mean"),
    highest=("score", "max")
)

print("브랜드별 요약:")
print(summary)
print("매체별 평균:")
print(df.groupby("media")["score"].mean())
```

코드 톨아보기

- **groupby("brand")** : 브랜드가 같은 행을 묶습니다. A의 평균은 85, B는 75, C는 90입니다.
- **agg의 이름 지정 집계** : 결과 열 이름과 원본 열, 집계 함수를 함께 지정합니다. count는 score의 결측값을 제외한 개수를 셉니다.
- **as_index=False** : 브랜드를 일반 열로 남겨 결과를 다루기 쉽게 합니다. 예제 응답 수는 브랜드별 2개이므로 일반화에는 제한이 있습니다.

[9차시] 표 결합과 막대그래프 비교

📌 핵심 개념 설명

분석 자료와 설명 자료가 별도 표에 있을 때 공통 키를 기준으로 결합할 수 있습니다. merge로 브랜드별 점수와 분류를 연결한 뒤 막대그래프로 비교합니다. 결합 키가 중복되면 행 수가 예상보다 늘어날 수 있으므로 관계를 확인합니다. 그래프의 영어 표기는 한글 글꼴 설정 없이 실행하기 위한 것입니다.

📄 전체 실습 코드

```
import pandas as pd
import matplotlib.pyplot as plt

scores = pd.DataFrame({"brand": ["A", "B", "C"],
                        "average": [85, 75, 90]})
info = pd.DataFrame({"brand": ["A", "B", "C"],
                     "category": ["음료", "과자", "식품"]})
merged = scores.merge(info, on="brand", how="left",
                      validate="one_to_one")

#데이터 연결의 공통 기준 열을 brand로, 기준이 되는 왼쪽 데이터인 scores의 모든 행은 무조건 보존,
#validate=one_to_one은 데이터가 정확히 1대1 관계로 연결되는지 검증

plt.figure(figsize=(8,5))
plt.bar(merged['brand'], merged['average'])
plt.title("Average survey score by brand")
plt.xlabel("Brand")
plt.ylabel("Score")
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
```

🔍 코드 톨아보기

- **left 결합** : 왼쪽 scores의 브랜드를 모두 유지합니다. 오른쪽에 대응하는 키가 없으면 category가 결측값이 됩니다.
- **validate="one_to_one"** : 두 표 모두 브랜드 키가 유일한지 검사합니다. 예제처럼 브랜드별 한 행인 표를 결합할 때 유용합니다.
- **막대그래프 해석** : 세로축을 0~100으로 고정하여 평균 차이를 비교합니다. C가 가장 높지만 이 가상 자료만으로 실제 광고 효과를 단정할 수 없습니다.